Name:- Aaryan N. Ruparane Roll No:- 5004 Div:- 5
branch:- AI&DS batch:- A1 SAP ID:- 60018250070

Experiment No 6

Aim:-

To understand and apply the basics of working with Pandas Series and DataFrames in Python focusing on fundamental operations useful in data science and Machine learning.

Theory:-

1) What is a Pandas Series? How is it different from a Numpy array?

=> A Pandas Series is a one dimensional labeled array capable of holding data of any type such as integers, floats, or strings. Each element has a corresponding label called an index, which can be integers or custom string labels. It is built on top of Numpy and is therefore fast and memory efficient. A Numpy array uses only integers position to access elements and does not carry an index label. A series adds a labeled index on top, which allows to access by name and enables automatic data alignment when two series are combined. Series also support missing value handling using NaN, which Numpy arrays do not provide by default.
for example:- If two series with different index labels are added, Pandas aligns them by label before performing the operation. This makes series

Sundaram
FOR EDUCATIONAL USE

more suitable than plain. Numpy arrays when working with real world datasets that may have gaps or irregular ordering.

25 How can you create a DataFrame from a Python dictionary? Give an example.

⇒ A DataFrame is a two dimensional table with labelled rows and columns. It can be created by passing a Python dictionary to pd.DataFrame(). The dictionary keys become the column names and the corresponding lists become the column data. If no index is specified, Pandas assigns an integer index starting from 0 automatically. All lists in the dictionary must have the same length, otherwise Pandas raises a ValueError. You can verify the shape and column names of the created DataFrame using df.shape and df.columns.

Eg:-

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [21, 22, 20],
    "Marks": [88, 75, 91]
}
df = pd.DataFrame(data)
print(df)
print("Shape:", df.shape)
print("Columns:", list(df.columns))
```

Sundaram
FOR EDUCATIONAL USE

output :

	Name	Age	Marks
0	Alice	21	88
1	Bob	22	75
2	Charlie	20	91

Shape : (3, 3)
Columns : ['Name', 'Age', 'Marks']

3) Explain the use of .loc[] and .iloc[] with suitable examples.

=> .loc[] is label based indexing. It selects rows and column using their assigned labels (index names and column names). Both the start and end labels are inclusive. If a label does not exist, Pandas raises a KeyError. It also supports boolean conditions for filtering rows.

.iloc[] is integer position based indexing. It selects rows and columns using their 0 based integer positions, following Python slicing rules where the end position is exclusive. It is useful when the index labels are not known or when you need positional access.

Eg :-
```
import pandas as pd
data = {
    "city": ["Mumbai", "Delhi", "Pune", "Chennai"],
    "Population": [7205, 3200, 906, 1160]
}
df = pd.DataFrame(data, index=["r1", "r2", "r3", "r4"])
```

Sundaram
FOR EDUCATIONAL USE

```
print("Using .loc[]:")
print(df.loc["r1":"r3", city])
print("Using .iloc[]:")
print(df.iloc[0:2,0])
```

Output:

```
Using .loc[]:
r1    Mumbai
r2     Delhi
r3      Pune
Name: city, dtype: str
Using .iloc[]:
r1    Mumbai
r2     Delhi
Name: City , dtype: str
```

4) How do you check the datatype of each column in a DataFrame?

⇒ The .dtypes attribute of a DataFrame returns the data type of every column. This is useful to confirm whether columns are stored as the correct type before performing any operations or building machine learning models.
Common dtypes returned are int64 for integer columns, float64 for decimal columns, and Object for string or mixed type columns. If a column has the wrong dtype, it can be converted using the .astype() method.
Eg: df['Age'].astype(float) Converts the Age column from int64 to float64

Sundaram®
FOR EDUCATIONAL USE

Eg:-
```
import pandas as pd

data = {
    "Product": ["Pen", "Book", "Bag"],
    "Price": [10.5, 150.0, 500.0],
    "Quantity": [100, 50, 30]
}
df = pd.DataFrame(data)
print(df.dtypes)
```

output:-
```
Product    str
Price      float64
Quantity   int64
dtype: object
```

5) Differentiate between head(), tail(), and sample() functions.

=> These three functions are used to view a subset of rows from a DataFrame. head() and tail() always return the same rows, while sample() returns a randomly selected set of rows that may differ on each call.

feature	head()	tail()	sample()
Returns	First n rows (default 5)	Last n rows (default 5)	n random rows (default 1)
Order	Always from the top	Always from the bottom	Random, changes each call

Sundaram
FOR EDUCATIONAL USE

Use case	Always from the top	Always from the Bottom	Random
Use case	Preview the start of data	Preview the end of data	Get a random sample.
Reproducible	Yes, always the same result	Yes, always the same result	No, varies each execution
Syntax Example	df.head(3)	df.tail(3)	df.sample(3)

6) Give applications of Pandas in data Science.

=> Pandas is widely used across data science tasks. Some key applications are listed below:

i) Data cleaning: Handles missing values using fillna() and dropna(), removes duplicate rows, and corrects inconsistent entries.

ii) Exploratory data Analysis: Provides statistical summaries through describe() and info(), helping understand the distribution and structure of data.

iii) Data Manipulation: Supports filtering rows with conditions, sorting by column values, grouping data using groupby(), and merging multiple DataFrames.

iv) Feature Engineering: Helps create new columns derived from existing ones and apply transformation functions to prepare data for Machine learning models.

v) Time Series Analysis: Supports date indexing, resampling, and rolling window operations, making it useful for analyzing data collected over time.

vi) Data Aggregation: Functions like groupby() combined with agg() allow summarizing large datasets by computing total, averages and counts across groups.

Conclusions:

In this Experiment, Pandas Series and DataFrame structures were studied along with their creation and core operations. Functions such as .loc[], .iloc[], .dtypes, head(), tail(), and sample() were applied on DataFrames to perform label based and position based data access. Pandas proves to be a powerful library for data manipulation and preprocessing, forming a foundational tool in any data science workflow.

Sundaram
FOR EDUCATIONAL USE